ASIAKASSOVELLUSTEN LIITTÄMINEN PALVELINSOVELLUKSIIN

Sivusto:	Xamk Learn	Tulostanut:	Laura Hietala
Opintojakso:	Sovellusohjelmointi 2	Tulostettu:	torstaina 26. joulukuuta 2024, 17.45
Kirja:	Asiakassovellusten liittäminen palvelinsovelluksiin		

SISÄLLYSLUETTELO

1. Aluksi
2. React-asiakassovelluksen toteutus
3. JWT-autorisointi
4. Käyttäjähallinta
5. Yhteenveto

1. ALUKSI

Tässä materiaalissa tutustaan ns. asiakassovellusten toteutukseen Reactilla niiden yhteistoimintaa palvelinsovellusten kanssa. Aiheen yhteydessä käsitellään myös tietoturvakysmyksiä, kuten autorisointi, autentikointi ja käyttäjähallinta. Perehdy materaaliin itsenäisesti ja omaan tahtiin.

Sisältönä on tekstin lisäksi myös videoita, joissa toteutetaan pienimuotoisia demosovelluksia liittyen sisältöihin tai havainnollistaan kehitystyötä muutoin. Voit tutustua demokoodeihin tarkemmin tämän GitHub-repositorion kautta. Voit vapaasti kokeilla koodien toimivuutta omalla laitteellasi, tehdä muutoksia jne. Tehokas tapa opiskella on kommentoida demokoodia samalla, kun katsot videoita.

Materiaalin sisäistämisen edeltävyysehtona on, että tunnet React-sovellusohjelmointia (Sovellusohjelmointi 1) vähintään perusteiden tasolla.

Materiaalin lopuksi on yhteenveto, jossa on kysymyksiä opiskeltaviin sisältöihin liittyen. Kun koet, että osaat vastata kaikkiin kysymyksiin, voit jatkaa opiskelua tekemällä materiaaliin liittyviä oppimistehtäviä 5-6.

2. REACT-ASIAKASSOVELLUKSEN TOTEUTUS

Tähän saakka olemme toteuttaneet palvelinsovelluksia itsenäisinä kokonaisuuksina ja testanneet toiminnallisuutta Postman-sovelluksella. Pelkkä REST API ei kuitenkaan ole riittävä, jos käyttäjäkohderyhmänä ovat ihmiskäyttäjät. Palvelinsovelluksen toiminnallisuuksien käyttöön tarvitaan siis havainnollisempi ja intuitiivisempi käyttöliittymä, ns. **client- eli asiakassovellus**.

Asiakassovelluksia voidaan toteuttaa melkeinpä millä tahansa ohjelmointikielellä tai -tekniikalla. Kun asiakassovellus tehdään REST API-palvelinsovellukseen, riittää, että käytetyssä tekniikassa on tuettuna http-tietoliikenneominaisuudet. Tyypillisesti asiakassovellukset ovat Javascript-selainsovelluksia, myös monet natiivisovellukset kuten älypuhelin-, äly-tv- tai työpöytäsovellukset voivat toimia asiakassovelluksena.

Seuraavalla videolla toteutetaan React-sovellus, joka toimii asiakassovelluksena aikaisemmin luotuun ostoslista-palvelinsovellukseen.



Videolla tulee esille ns. **CORS**-käytännön (Cross-Origin Resource Sharing) asettamat rajoitukset asiakas- ja palvelinsovelluksen yhteistyölle. Käytäntö on perusteltu turvallisuussyillä. Selainten verkko-ominaisuuksiin (kuten demolla käytetty Fetch API tai sen edeltäjä XMLHttpRequest) on toteutettu rajoitus, joka estää palvelinpyyntöjen tekemisen muualle kuin samalle palvelimelle, josta js-koodi on selaimeen ladattu (ns. same-origin policy).

CORS-käytäntöön voidaan kuitenkin tehdä poikkeus palvelinsovelluksessa ja määritellä esim. tietyt osoitteet, joista palvelin sovellukseen voidaan olla yhteydessä CORS-rajoituksista huolimatta. Node/Express-palvelinsovelluksissa tämä tehdään tyypillisesti cors-npm-paketin avulla. Voit halutessasi perehtyä CORS:iin syvemmin esim. MDN-dokumentaatiosta .

3. JWT-AUTORISOINTI

Tässä osuudessa aloitamme perehtymisen asiakas- ja palvelinsovellusten turvallisuuskysymyksiin. Kuten olemme jo oppineet, palvelinsovellukset toimivat oletuksena samalla tavoin rippumatta siitä millainen asiaksovellus niitä kutsuu. Ostoslista-sovelluksemme palvelinosaa voi käyttää yhtä lailla omalla React-sovelluksella kuin Postmanilla tms. Edellä esitellyillä CORS-säännöillä voimme jossain määrin rajoittaa selaimessa toimivien asiakassovellusten käyttöä, mutta tämä ei täysin estä palvelinsovelluksen käyttöä muualta, esim. Postmanilla.

Kaikkien vapaasti ja mistä tahansa avoimesti käytettävä palvelinsovellus on harvoin toteutuksen tarkoitus. Palvelinsovellukset tuleekin usein toteuttaa siten, että palvelimen toimintoihin pääseekin vain tietyillä asiakassovelluksilla. Tämä toteutetaan siten, että asiakassovellus **autorisoidaan** eli valtuutetaan käyttämään palvelinresursseja.

Yksi yleisimmistä tekniikoista asiakas- ja palvelinsovelluksen keskinäiseen autorisointiin on **JWT (JSON Web Token)**. JWT on avoin standardi kahden osapuolen esim. (asiakas- ja palvelinsovelluksen) turvalliseen viestinvälitykseen. JWT:n teknisenä ratkaisuna on ns. token eli merkkijono, joka allekirjoitetaan digitaalisesti. Allekirjoitus tehdään tyypillisesti salaisella avaimella, jonka vain valtuutuksen tehnyt taho (esim.

palvelinsovellus) tietää. Kun valtuutuksen saanut taho (esim. asiakassovellus) lähettää pyynnön yhteydessä tokenin, jonka allekirjoitus voidaan varmistaa salaisella avaimella, viestivät osapuolet voivat luottaa toisiinsa. Lisätietoja JWT:stä https://jwt.io/introduction.

Videolla toteutetaan ostoslista-sovellukseen asiakassovelluksen autorisointi perustuen JWT:n käyttöön. Lisäksi kirjoitetaan myös pieni apuohjelma tokenien luontiin.



TURVALLISUUSHUOMIO! Kuten videon lopussakin todetaan, tämä versio JWT:n käytöstä sisältää tietoturvariskin siinä, että token on osana asiakassovelluksen lähdekoodeja. Token on siis helposti kopioitavissa ja kuka/mikä tahansa voi esiintyä valtuutettuna sovelluksena niin kauan, kun palvelimella määritelty avain ei muutu. Näin ei siis tehdä "oikeassa maailmassa" varsinkaan selainsovellusten (kuten React) kanssa toimiessa. Seuraavalla videolla jatketaan toteutusta ja opitaan turvallisempi tapa luoda token ohjelmallisesti perustuen onnnistuneeseen kirjautumiseen.

Tiedoksi vielä: Jos kopiot demon koodit omaan GitHubiisi, voit saada turvallisuushuomautuksen juuri tästä syystä. Ilmoituksesta ei pidä hätääntyä, kun tiedät mistä tässä tapauksessa on kyse. Mutta älä muutoin laita tokeneita GitHubiin!

4. KÄYTTÄJÄHALLINTA

Koska palvelinsovelluksilla on tyypillisesti useita, jopa yhtäaikaisia käyttäjiä, tulee **käyttäjähallinta** (joskus identiteetin hallinta, IM tai pääsyn hallinta, IAM) keskeiseen rooliin jo hyvinkin pienimuotoisissa sovelluksissa. Käyttäjähallinnassa on tyypillisesti kaksi merkittävää kokonaisuutta: **autorisointi** (käyttöikeuden valtuuttaminen) ja **autentikointi** (käyttäjän tunnistaminen ja yksilöinti)

Käyttäjähallinnan työnkulku on tyypillisesti seuraava: Aluksi käyttäjä tunnistautuu (autentikointi) sovellukseen jollain tavoin, esim. käyttäjätunnuksella ja salasanalla. Käyttäjän identititetti yksilöidään käyttäjätunnuksella ja vahvistetaan salasanalla. Onnistuneen autentikoinnin jälkeen käyttäjä saa valtuutuksen (autorisointi) hänelle sallittuihin resursseihin. Käyttäjiä voi olla olla eri tasoisilla valtuutuksilla (vain luku -oikeus, muokkausuoikeudet jne.) ja ne voidaan kytkeä käyttäjätietohin, koska autentikointi erottaa käyttäjät toisistaan.

Videolla toteutetaan käyttäjähallinta ostoslista-sovellukseen siten, että sovellusta voi käyttää vasta kirjautumalla oikealla tunnuksella ja salasanalla. Lisäksi tehdään käyttäjätietojen tallentaminen tietokantaan ja käsitellään erinäisiä turvallisuuteen liittyviä kokonaisuuksia (esim. salasanatiivisteet)



5. YHTEENVETO

Tässä materiaalissa olemme opiskelleet asiakassovelluksen toteuttamista osaksi palvelinsovellusta. Lisäksi olemme perehtyneet autentikoinnin ja autorisoinnin perusteisiin sekä toteuttaneet käyttäjähallinnan käytännössä. Tässä vaiheessa sinulla pitäisi olla osaamista seuraavista asiakokonaisuuksista.

- Mikä on asiakassovellus?
- Kuinka React-asiakassovelluksella ollaan yhteydessä palvelinsovellukseen/REST API:iin?
- Mitä tarkoittaa CORS? Kuinka se tulee ottaa huomioon asiakas-/palvelinsovellusten toteutuksessa?
- Mitä tarkoittaa käyttäjähallinnan käsitteet autorisointi ja autentikointi? Kuinka nämä eroavat toisistaan?
- Mikä on JWT eli JSON Web Token?
- Kuinka salasanatiivisteet (hash) tehdään? Kuinka käyttäjätietojen (käyttäjätunnus ja salasana) oikeellisuus tarkastetaan turvallisesti?

Mikäli joku näistä on sinulle vielä epäselvää, kertaa materiaalia. Muutoin voit ryhtyä tekemään oppimistehtäviä 5-6 ja opiskella aihetta lisää käytännön tekemisen kautta.